Wenbo: Scaling Agent Reasoning via Dynamic DAG Scheduling

Feiyang Chen
feiyang73@sjtu.edu.cn
Shanghai Jiao Tong Univ.
Shanghai, China

Junqing Yang
jy5675@nyu.edu.cn
New York Univ.
New York, United States

Wentao Zhang
zhang_wt@sjtu.edu.cn
Shanghai Jiao Tong Univ.
Shanghai, China

Zheng Wang
wzheng@sjtu.edu.cn
Shanghai Jiao Tong Univ.
Shanghai, China

Li Pan
panli@sjtu.edu.cn
Shanghai Jiao Tong Univ.
Shanghai, China

Jianhua Li
lijh888@sjtu.edu.cn
Shanghai Jiao Tong Univ.
Shanghai, China

Abstract

Large-scale LLM agent serving suffers from resource fragmentation because existing systems assign each task a fixed bundle of GPU and CPU resources. We present **Wenbo**, a DAG-centric execution system that builds executable DAGs from agent reasoning graphs and dynamically schedules their computation nodes across shared GPUs and CPUs through a Director, Scheduler, and Workers. Our theoretical analysis shows that the potential speedup depends on the balance between resource availability and workload demands *For example, with one GPU and 1,024 CPUs, the potential speedup can reach* **1,024**×. We also introduce **ARGB**, an agent reasoning graph benchmark comprising real-world workloads (**ARGB-R**) and configurable synthetic workloads (**ARGB-S**). Experiments show that Wenbo achieves 2–4× higher throughput than the state-of-the-art method Flash-Searcher on real-world workloads, with even much larger gains on synthetic workloads. Our code is available at https://github.com/wenbo-project/wenbo.

Keywords

Agent systems; DAG-based scheduling; Heterogeneous computing

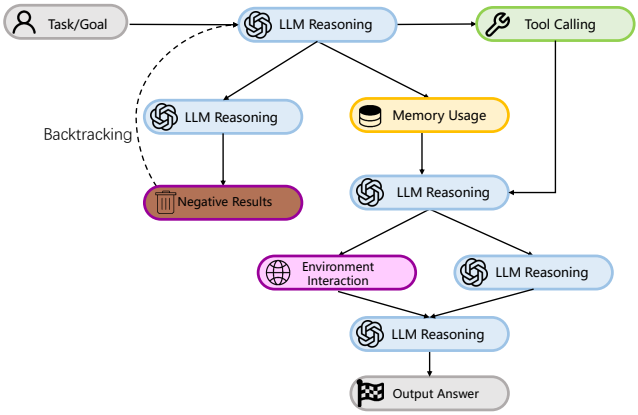


Figure 1: Graph abstraction for LLM agent reasoning.

1 Introduction

The remarkable reasoning capabilities of large language models (LLMs) have made LLM-based agents a powerful paradigm for solving complex tasks that require planning, tool use, memory access, and interaction with external environments. By combining LLM inference with external tools and control logic, agent frameworks such as LangChain, AutoGen, and BabyAGI support applications ranging from code generation to multi-step research assistance [3, 8, 15]. As these applications evolve toward large-scale services with massive numbers of concurrent agent tasks, efficiently managing the computation and resources required by agent reasoning becomes an increasingly important systems challenge.

However, existing agent systems typically adopt a task-centric execution model, in which an incoming agent task is assigned a fixed resource bundle, such as GPU and CPU resources, and retains these resources throughout its lifetime [7]. This model is ill-suited to large-scale agent workloads because agent execution consists of heterogeneous computation stages with different resource requirements. LLM inference is primarily GPU-intensive, whereas tool invocation, retrieval, data processing, and orchestration often rely on CPUs. These stages alternate dynamically throughout an agent's execution, leaving some allocated resources idle while others are actively used. Such coarse-grained resource ownership causes substantial resource fragmentation and limits cluster-wide utilization. Consequently, scaling agent services requires an execution model that can decouple individual computation steps from the resources assigned to an entire task.

Meanwhile, recent advances in LLM reasoning have demonstrated the effectiveness of graph-based approaches, exemplified by Graph-of-Thoughts, which organize reasoning into interconnected thoughts rather than a single linear chain [18]. As shown in Figure 1, an agent reasoning process can also be naturally represented as a reasoning graph, where each node corresponds to an atomic computation step, such as LLM generation, tool invocation, retrieval, verification, or data processing, and each edge represents a dependency between steps. Unlike a static workflow, this reasoning graph may evolve dynamically as intermediate LLM outputs and tool results determine subsequent computation. Nevertheless, at any point during execution, portions of the graph whose dependencies are resolved can be materialized into executable directed acyclic graphs (DAGs). This representation exposes the dependency structure and available parallelism of agent execution, providing a foundation for fine-grained scheduling across heterogeneous resources.

Motivated by this observation, we present **Wenbo**, a DAG-centric execution system for massive agent reasoning workloads. As shown in Figure 2, Wenbo decouples agent reasoning from resource management through three major components. The *Director* transforms

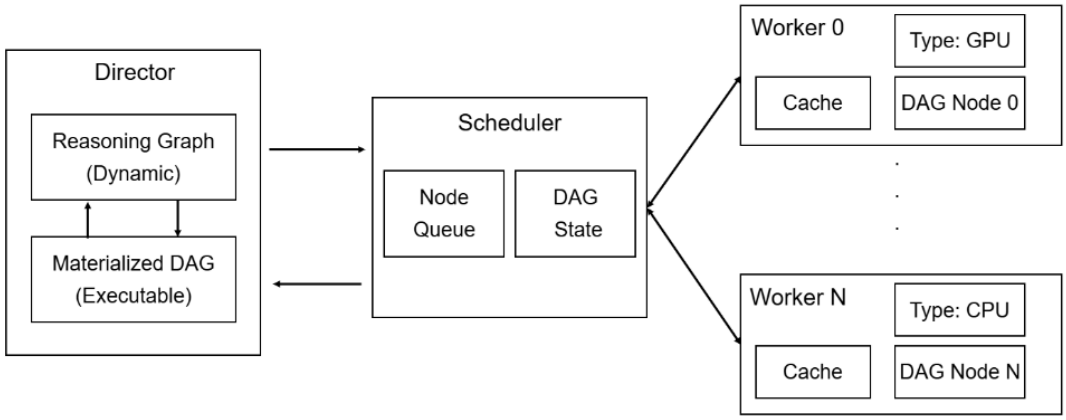


Figure 2: Architecture of Wenbo.

incoming agent tasks into dynamic reasoning graphs and incrementally materializes executable DAG flows according to intermediate reasoning states. The *Scheduler* maintains a global view of executable computation nodes across all active DAGs and dispatches them to heterogeneous GPU and CPU workers based on their computation requirements. The *Workers* execute individual computation nodes and exchange intermediate results through a shared execution layer. By separating reasoning orchestration, resource scheduling, and hardware execution, Wenbo enables fine-grained resource sharing and scalable parallel execution across heterogeneous computing resources.

We further develop a theoretical framework to characterize the throughput benefits of node-level scheduling compared with traditional task-level scheduling. Our analysis reveals that the achievable improvement is determined by the alignment between system resource composition and workload composition. When task-level scheduling binds heterogeneous resources together, abundant resources can remain idle if their corresponding computation phase is not active. In contrast, node-level scheduling can dynamically redistribute these resources across independent computation nodes from different tasks. In the idealized setting, the throughput improvement can approach the degree of resource imbalance, revealing when DAG-centric execution provides the largest benefits.

Our contributions are summarized as follows.

- **DAG-centric execution abstraction for massive agent serving.** We identify resource fragmentation caused by task-level resource ownership as a fundamental scalability bottleneck in agent serving. We introduce a DAG-centric execution model that enables fine-grained scheduling of agent reasoning steps across heterogeneous resources.
- **Wenbo system architecture.** We design and implement Wenbo, a scalable agent execution system composed of a Director, Scheduler, and Workers. The system supports dynamic reasoning graph generation, online DAG refinement, global DAG scheduling, and heterogeneous GPU/CPU execution.
- **Theoretical analysis of node-level scheduling.** We develop a throughput model that characterizes the advantage of node-level scheduling over task-level scheduling. The analysis reveals how performance improvement depends on resource heterogeneity, workload characteristics, and the available parallelism within reasoning graphs.
- **Experimental evaluation.** We evaluate Wenbo using synthetic and realistic agent workloads. The results demonstrate significant throughput improvements over task-level scheduling and validate the conditions under which DAG-centric execution provides substantial gains.

2 System Design

Wenbo introduces a graph-driven execution framework for massive agent reasoning workloads. Instead of assigning resources to an entire agent task, Wenbo exposes the internal computation structure of agent reasoning and enables fine-grained execution at the granularity of individual computation nodes. Each agent task is represented as a dynamic reasoning graph, which is incrementally materialized into executable DAG flows. These DAG nodes can then be independently scheduled across a shared pool of heterogeneous workers, allowing GPU-intensive and CPU-intensive operations from different tasks to share cluster resources efficiently.

As shown in Figure 2, Wenbo consists of three major components: the *Director*, which manages the evolution of reasoning graphs and generates executable computation plans; the *Scheduler*, which performs global dependency-aware node scheduling across heterogeneous workers; and the *Workers*, which execute individual computation nodes and provide runtime feedback.

2.1 Director: Dynamic DAG Generation

The Director serves as the reasoning control plane of Wenbo. Unlike a traditional workflow planner that generates a fixed execution graph before runtime, the Director manages a dynamically evolving reasoning process, where future computation steps may depend on intermediate execution results.

Given an incoming agent task, the Director first analyzes the task objective and constructs an initial reasoning graph. Each node

in the reasoning graph represents an atomic reasoning or computation step, such as LLM inference, tool invocation, retrieval, verification, or data processing, while edges represent logical dependencies among these steps. The reasoning graph captures the semantic structure of how the agent solves a problem, rather than only the execution order.

To enable distributed execution, the Director incrementally materializes executable DAG flows from the current reasoning graph. During this process, the Director identifies executable computation nodes, determines their dependency constraints, and annotates each node with execution requirements, including computation type and resource preference. The generated DAG therefore serves as an execution representation of the current reasoning state, which can be directly consumed by the Scheduler for distributed execution.

The reasoning graph is continuously updated throughout task execution. Intermediate results produced by Workers may reveal new information, invalidate previous assumptions, or introduce additional reasoning requirements. Upon receiving execution feedback, the Director evolves the reasoning graph by creating new nodes, refining dependencies, expanding new execution branches, or terminating unnecessary computation paths. It then incrementally updates the corresponding executable DAG flows. This dynamic graph evolution enables Wenbo to support open-ended agent reasoning, where the complete execution plan is not required to be known before execution begins.

In addition to reasoning graph management, the Director maintains the global context state of each agent task. It manages the lifecycle of intermediate data generated across different computation nodes, tracks dependencies among data objects, and provides required context when new reasoning steps are generated. By maintaining both the evolving reasoning graph and the associated task context, the Director bridges high-level agent reasoning with low-level distributed execution.

2.2 Scheduler: DAG-aware Node Scheduling

The Scheduler is the execution control plane of Wenbo. Its primary responsibility is to map executable DAG nodes to appropriate heterogeneous computing resources. Based on the node types and resource requirements specified by the Director, the Scheduler dispatches each node to a suitable GPU or CPU worker. It continuously monitors resource availability and execution progress, allowing nodes from the same DAG as well as nodes from different DAGs to execute concurrently. In this way, the Scheduler separates the logical reasoning plan from its physical execution and enables Wenbo to efficiently utilize a heterogeneous cluster.

The Scheduler also provides elastic fault tolerance for distributed execution. Workers may fail due to hardware failures, resource exhaustion, network errors, or other runtime issues. When a worker failure is detected, the Scheduler identifies the affected nodes and reschedules retryable tasks to other suitable workers without restarting the entire DAG. Since the execution state and intermediate results of completed nodes are preserved, Wenbo can recover at the granularity of individual DAG nodes and avoid unnecessary recomputation. This design allows the system to continue making progress despite failures in the underlying worker pool.

In addition, the Scheduler maintains the temporary execution state of active DAGs. This state includes node execution status, assigned workers, intermediate execution results, and other information required to coordinate ongoing computation. The Scheduler continuously communicates this execution state to the Director, allowing the Director to observe intermediate outcomes and provide new instructions when the reasoning plan needs to evolve. For example, the Director may introduce new nodes or modify subsequent execution based on intermediate results. The Scheduler then incorporates these updates into the current execution state and continues scheduling the resulting DAG. Thus, the Scheduler serves as the bridge between the evolving logical reasoning process managed by the Director and the physical execution performed by the Workers.

2.3 Worker: Heterogeneous Node Execution

Workers are the execution layer of Wenbo. Their primary responsibility is to execute individual DAG nodes dispatched by the Scheduler. Upon receiving a task, a worker obtains the required inputs and execution context, performs the corresponding computation, and produces the node output. Workers are agnostic to the global reasoning process and focus on reliably executing the specific computation assigned to them.

Workers also manage their local computing resources, including GPUs, CPUs, memory, models, and runtime environments. Each worker maintains the state of its available resources and exposes this information to the Scheduler, allowing the Scheduler to make resource-aware placement decisions. For GPU workers, this may include managing model instances, GPU memory, and inference environments; for CPU workers, it may include managing CPU capacity, memory, and execution environments. This local resource management hides hardware-specific details from the higher-level scheduling layer.

Finally, Workers continuously provide execution feedback to the Scheduler. In addition to the output of each DAG node, this feedback includes execution status, resource utilization, execution time, failures, and other runtime information. The Scheduler uses this information to update the current DAG execution state and make subsequent scheduling decisions, while the Director can use relevant execution outcomes to determine whether the reasoning DAG should be further updated. Workers therefore close the execution loop between physical computation, scheduling, and the evolution of agent reasoning.

3 System Analysis

3.1 Theorical Analysis of Scheduling

Consider a computing pool with N_{GPU} identical GPUs and N_{CPU} identical CPUs. An infinite stream of homogeneous agent tasks arrives. Each task is pre-converted into a DAG of computational nodes. Every node is either a GPU node or a CPU node and has unit execution time. The total GPU work and total CPU work of one task are denoted by $\tau_{\text{GPU}} > 0$ and $\tau_{\text{CPU}} > 0$, respectively. There are no cross-task dependencies, and nodes from different tasks may execute concurrently, provided intra-task dependencies are respected.

Definition 1 (Task-level parallelism, baseline). *The system maintains a queue of tasks. At most $K = \min(N_{\text{GPU}}, N_{\text{CPU}})$ tasks can be in service simultaneously. Each admitted task is allocated exactly one GPU and one CPU, and it holds both resources until completion. Its DAG nodes are scheduled on this fixed resource pair as soon as their intra-task dependencies allow.*

Lemma 1 (Execution time and throughput of task-level parallelism). *Let T_{task} be the minimum wall-clock execution time of one task under Definition 1. Then*

$$\max(\tau_{\text{GPU}}, \tau_{\text{CPU}}) \;\le\; T_{\text{task}} \;\le\; \tau_{\text{GPU}} + \tau_{\text{CPU}}. \tag{1}$$

Moreover, the steady-state throughput of task-level parallelism is

$$\Theta_{\text{task}} \;=\; \frac{K}{T_{\text{task}}}, \tag{2}$$

where $K = \min(N_{\text{GPU}}, N_{\text{CPU}})$.

Proof of Lemma 1. A task receives exactly one GPU and one CPU. The GPU must spend a total of τ_{GPU} busy time and the CPU must spend a total of τ_{CPU} busy time. Even with perfect overlap, the task cannot finish before both totals have been served, hence before $\max(\tau_{\text{GPU}}, \tau_{\text{CPU}})$.

Conversely, because there are only two resources and the task is a DAG, at least one of the two resources can always be busy until the task completes. Indeed, if work remains, any source node in the remaining DAG has all dependencies satisfied and is ready to execute on either the GPU or the CPU. Therefore the makespan cannot exceed the total busy time $\tau_{\text{GPU}} + \tau_{\text{CPU}}$.

Since K tasks are in service simultaneously and each completes after T_{task}, the departure rate is K/T_{task}. □

Definition 2 (Node-level parallelism). *All nodes of all tasks are submitted to a global scheduler that can map any GPU node to any idle GPU and any CPU node to any idle CPU as soon as its intra-task dependencies are met. There is no restriction that a task must be assigned to a fixed resource pair, and nodes from different tasks may be interleaved freely.*

Lemma 2 (Ideal throughput of Node-level parallelism). *Under Definition 2, zero scheduling overhead, and perfect cross-task pipelining, the maximal system throughput is*

$$\Theta_{\text{node}} \;=\; \min\!\left(\frac{N_{\text{GPU}}}{\tau_{\text{GPU}}},\, \frac{N_{\text{CPU}}}{\tau_{\text{CPU}}}\right). \tag{3}$$

Proof of Lemma 2. Let ρ denote the steady-state task completion rate. Each completed task consumes τ_{GPU} GPU busy time and τ_{CPU} CPU busy time. Therefore

$$\rho\,\tau_{\text{GPU}} \le N_{\text{GPU}}, \qquad \rho\,\tau_{\text{CPU}} \le N_{\text{CPU}}.$$

Hence

$$\rho \le \min\!\left(\frac{N_{\text{GPU}}}{\tau_{\text{GPU}}},\, \frac{N_{\text{CPU}}}{\tau_{\text{CPU}}}\right).$$

Under the ideal assumptions, the bottleneck resource can be kept fully occupied by interleaving nodes from different tasks, so the upper bound is achievable. □

Theorem 3 (Throughput improvement factor). *Let*

$$\lambda = \frac{\Theta_{\text{node}}}{\Theta_{\text{task}}}$$

be the throughput improvement factor of Node-level parallelism over task-level parallelism. Then

$$\lambda_{\min} \;\le\; \lambda \;\le\; \lambda_{\max}, \tag{4}$$

where

$$\lambda_{\min} \;=\; \frac{\max(\tau_{\text{GPU}}, \tau_{\text{CPU}})}{\min(N_{\text{GPU}}, N_{\text{CPU}})}\, \min\!\left(\frac{N_{\text{GPU}}}{\tau_{\text{GPU}}},\, \frac{N_{\text{CPU}}}{\tau_{\text{CPU}}}\right), \tag{5}$$

$$\lambda_{\max} \;=\; \frac{\tau_{\text{GPU}} + \tau_{\text{CPU}}}{\min(N_{\text{GPU}}, N_{\text{CPU}})}\, \min\!\left(\frac{N_{\text{GPU}}}{\tau_{\text{GPU}}},\, \frac{N_{\text{CPU}}}{\tau_{\text{CPU}}}\right). \tag{6}$$

Proof. Let

$$K = \min(N_{\text{GPU}}, N_{\text{CPU}}), \qquad D = \min\!\left(\frac{N_{\text{GPU}}}{\tau_{\text{GPU}}},\, \frac{N_{\text{CPU}}}{\tau_{\text{CPU}}}\right).$$

From Lemma 1 and Lemma 2,

$$\lambda = \frac{\Theta_{\text{node}}}{\Theta_{\text{task}}} = \frac{D}{K/T_{\text{task}}} = \frac{T_{\text{task}}}{K}\, D.$$

By Lemma 1, Eq. (1),

$$\max(\tau_{\text{GPU}}, \tau_{\text{CPU}}) \;\le\; T_{\text{task}} \;\le\; \tau_{\text{GPU}} + \tau_{\text{CPU}}.$$

Because $K > 0$ and $D > 0$, multiplying the latter inequality by D/K preserves the ordering:

$$\frac{\max(\tau_{\text{GPU}}, \tau_{\text{CPU}})}{K}\, D \;\le\; \frac{T_{\text{task}}}{K}\, D \;\le\; \frac{\tau_{\text{GPU}} + \tau_{\text{CPU}}}{K}\, D.$$

Substituting the definitions of K and D yields exactly (5)–(6). □

3.2 Quick Estimation of Throughput Gain

Recall from Theorem 3 that the throughput gain of node-level parallelism over task-level parallelism is $\lambda \approx D\,T_{\text{task}}/K$. Using the optimistic estimate $T_{\text{task}} \approx \max(\tau_{\text{GPU}}, \tau_{\text{CPU}})$, this expression simplifies to a comparison between the GPU/CPU device imbalance and the GPU/CPU workload imbalance. This gives two practical estimators.

Case 1: Device counts and task workloads are known. In this case, substituting the optimistic estimate into $\lambda \approx D\,T_{\text{task}}/K$ yields:

$$\lambda \;\approx\; \min\left(\max\left(\frac{N_{\text{GPU}}}{N_{\text{CPU}}}, \frac{N_{\text{CPU}}}{N_{\text{GPU}}}\right), \max\left(\frac{\tau_{\text{GPU}}}{\tau_{\text{CPU}}}, \frac{\tau_{\text{CPU}}}{\tau_{\text{GPU}}}\right)\right).$$

The gain is limited by whichever imbalance is smaller: a skewed device pool helps only if task workloads are skewed in the same direction. *Example:* With 1000 GPUs and 100 CPUs, and tasks requiring 100 units of GPU work and 1 unit of CPU work, the device imbalance is 10 and the workload imbalance is 100, so the potential gain is about 10×.

Case 2: Only device counts are known. Before deployment, the workload imbalance may be unknown. In this case, the device imbalance alone gives a rough upper-bound estimate:

$$\lambda \;\lesssim\; \frac{\max(N_{\text{GPU}}, N_{\text{CPU}})}{\min(N_{\text{GPU}}, N_{\text{CPU}})}.$$

This estimate is meaningful only if future tasks demand the abundant resource proportionally; otherwise the actual gain will be closer to 1×. *Example:* With 1000 GPUs and 1 CPU, the device imbalance is 1000, so the potential gain can be up to 1000×, provided future tasks are indeed GPU-heavy.

In sum, the key takeaway is that the more the available GPU/CPU resource supply mismatches the GPU/CPU workload demand of agent tasks, the greater the throughput gain of Wenbo over task-level parallelism; in other words, the degree of imbalance between resource supply and task demand determines how much Wenbo can potentially improve.

4 ARGB: Agent Reasoning Graph Benchmark

We introduce **ARGB** (Agent Reasoning Graph Benchmark) to standardize agent reasoning workloads and evaluate large-scale agent reasoning systems. ARGB represents each task as a directed graph, where vertices denote atomic operations, such as LLM inference and tool invocation, and edges encode dependencies. Each node specifies its resource type (GPU or CPU), memory requirements, and execution latency. ARGB comprises two subsets: **ARGB-R**, derived from public task datasets, and **ARGB-S**, generated synthetically using a parameterized generator.

4.1 ARGB-R: Public Task-Derived Dataset

ARGB-R is derived from four public agent reasoning task datasets: StrategyQA [4], MuSiQue [13], ToolBench/ToolLLM [11], and ALFWorld [12], covering factual reasoning, multi-hop evidence reasoning, tool use, and embodied interaction, respectively. Table 1 summarizes their graph statistics and resource compositions. This benchmark is constructed in three steps.

First, we extract task-specific information from each dataset, including questions and supporting facts from StrategyQA, questions and evidence passages from MuSiQue, task descriptions, available tools, and invocation traces from ToolBench, and goals, environment descriptions, and action sequences from ALFWorld.

Second, we provide the extracted information to `deepseek-v4-flash` and prompt it to construct a DAG representing each task's reasoning and execution dependencies. Subtasks become operation nodes, and directed edges encode prerequisite relationships. LLM-intensive operations, such as reasoning, planning, and answer generation, are assigned to GPU resources, while tool invocations, environment interactions, and deterministic processing are assigned to CPU resources.

Finally, we normalize the generated graphs into a unified JSON format. Each graph records its source dataset, each node contains an identifier, execution prompt, and resource label, and each edge specifies its source and destination nodes. This representation enables consistent workload processing and scheduling across different task domains.

4.2 ARGB-S: Parameterized Synthetic Dataset

ARGB-S uses a parameterized generator to produce reproducible DAG workloads with configurable graph structures, resource requirements, and task concurrency. It shares the graph representation and JSON schema of ARGB-R, enabling controlled evaluation of scheduling behavior under diverse workload conditions.

The generator varies the number of layers, layer widths, and connectivity parameters to produce DAGs with different sizes, dependency patterns, and degrees of parallelism. With uniform node execution times, these workloads isolate the effects of graph topology and dependency constraints, revealing how effectively schedulers exploit intra-task parallelism.

The generator supports CPU and GPU nodes with configurable proportions. Since each node can execute only on a compatible resource, resource contention may delay ready nodes even when their execution times are identical. These workloads evaluate how schedulers coordinate dependency readiness with resource availability and how resource composition affects scheduling performance.

The generator also produces batches of independent DAGs that share a common pool of CPU and GPU resources. Varying the number of concurrent DAGs controls inter-task contention and the number of competing ready nodes. The scheduler must allocate limited heterogeneous resources across tasks while respecting their dependency constraints. These workloads evaluate inter-task resource allocation and scheduling performance as concurrency increases, reflecting scenarios in which multiple agent reasoning tasks execute simultaneously.

5 Experimental Evaluation

We evaluate Wenbo on real-world and synthetic workloads, focusing on throughput, device utilization, and tail latency under mixed-priority workloads.

5.1 Experimental Setup

Workload Configuration. All experiments use the datasets introduced in ARGB. For simplicity, we assign every node an execution cost of one time unit, regardless of device type, and ignore communication and other overheads. Within each comparison, all methods process the same complete dataset in the same task order and run until all inference tasks finish.

Baselines. We compare methods that support DAG-based reasoning workflows. Flash-Searcher [10] is a state-of-the-art approach to DAG-based agent reasoning, originally designed for single-task execution. We adapt it for parallel workloads by statically assigning one GPU and one CPU to each reasoning task and reserving both resources for the lifetime of its DAG.

5.2 Evaluation on ARGB-R

5.2.1 ***Throughput Scaling.*** Figures 3 and 4 show throughput as the numbers of CPUs and GPUs vary, respectively, while the other device type is fixed. We can find the following three key observations.

First, Wenbo achieves 2×–4× higher throughput than Flash-Searcher across all four datasets under identical resource configurations. The reason is that: by scheduling ready nodes across DAGs, Wenbo dynamically shares resources, whereas Flash-Searcher reserves them even when tasks await dependencies. Second, adding resources of only one device type barely improves Flash-Searcher's throughput, whereas Wenbo benefits from additional resources in most configurations. This further shows that by decoupling resource allocation from task lifetime, Wenbo exposes more of the available node-level parallelism. Third, Wenbo's throughput gains eventually plateau as additional resources shift the bottleneck to the other device type.

Table 1: Task sources, graph structures, and resource composition of ARGB-R.

Dataset	Task Type	Graphs	Nodes	Edges	Avg. Depth	GPU/CPU Ratio
StrategyQA	Factual reasoning and Boolean QA	2,061	7,642	6,090	2.67	73.6% / 26.4%
MuSiQue	Multi-hop evidence reasoning	19,938	95,044	85,668	4.25	57.3% / 42.7%
ToolBench	Tool selection and invocation	26,999	124,408	131,936	3.20	41.1% / 58.9%
ALFWorld	Embodied planning and interaction	6,374	76,455	79,065	10.69	44.4% / 55.6%
Total	Four public task sources	55,372	303,549	302,759	4.42	47.8% / 52.2%

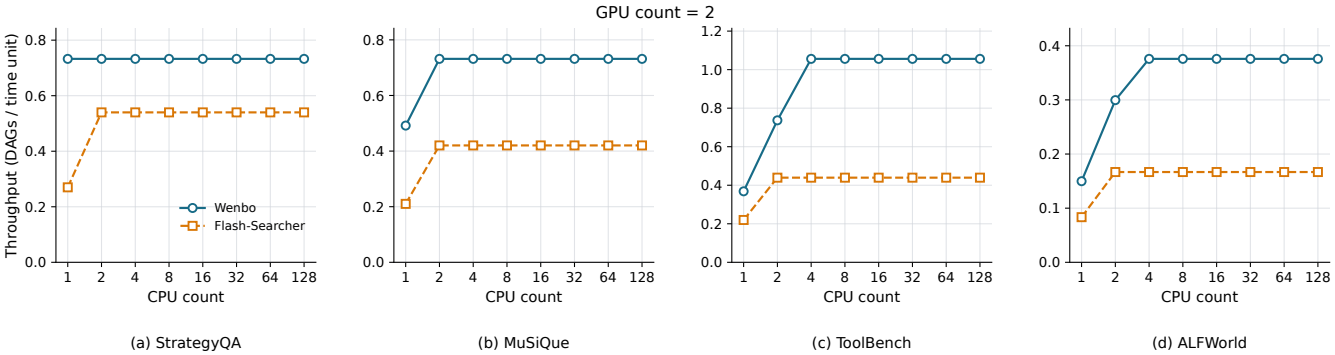


Figure 3: Average throughput on the ARGB-R benchmark, with the number of GPU slots fixed at 2.

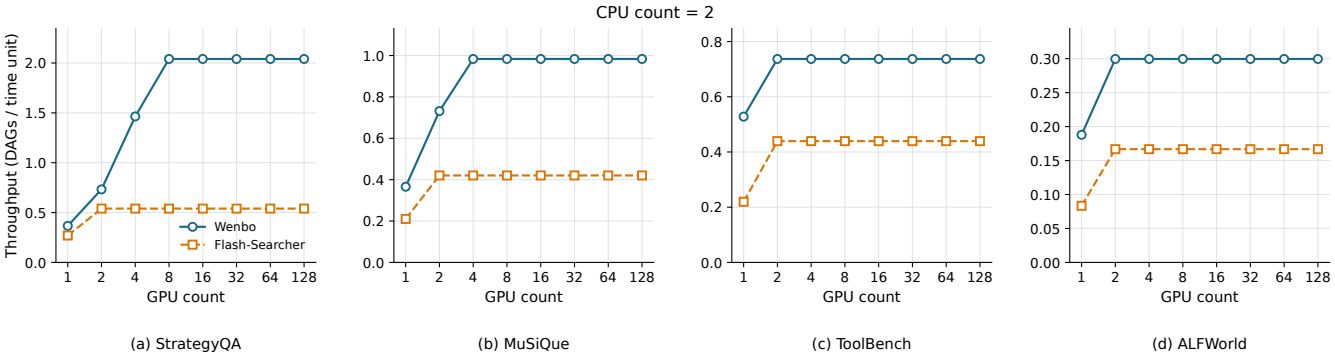


Figure 4: Average throughput on the ARGB-R benchmark, with the number of CPU slots fixed at 2.

This is because that the maximum gain from node-level over task-level parallelism is bounded by the smaller of the resource imbalance ratio and workload imbalance ratio. This behavior is consistent with the resource-capacity constraints established in our theoretical analysis.

5.2.2 ***Device Utilization.*** Table 2 reports GPU idle rates with the number of GPUs fixed at two while varying the number of CPUs.[1] We can clearly see that Wenbo substantially reduces GPU idle rates across all datasets and configurations. Unlike Flash-Searcher, which reserves GPUs throughout DAG execution, Wenbo releases devices immediately after node completion, allowing other ready nodes to use them while their original DAGs await dependencies. In addition, we also find that the impact of CPU scaling on Wenbo's GPU utilization varies across datasets, reflecting differences in CPU workload demands.. On StrategyQA and MuSiQue, GPU idle rates remain nearly unchanged, as GPUs are already highly utilized. In contrast, ToolBench and ALFWorld have heavier CPU workloads, so additional CPU resources accelerate CPU-bound nodes and expose subsequent GPU-bound nodes earlier. These results highlight how dependency-aware node-level scheduling improves GPU utilization by coordinating heterogeneous resources.

5.2.3 ***Priority Scheduling under Mixed Workloads.*** Following [5], we evaluate foreground latency under Poisson arrivals using two metrics: P99 first-service waiting time (from arrival to first-node execution) and P99 end-to-end response time (from arrival

[1]The CPU-scarce scenario can be obtained by swapping the two device types, yielding similar conclusions. Due to space constraints, we focus on the GPU-scarce scenario in the remaining experiments.

Table 2: GPU idle rates (%) on ARGB-R with the number of GPUs fixed at 2.

#CPUs	StrategyQA			MuSiQue			ToolBench			ALFWorld		
	Flash-Searcher	Wenbo	Imp.↑	Flash-Searcher	Wenbo	Imp.↑	Flash-Searcher	Wenbo	Imp.↑	Flash-Searcher	Wenbo	Imp.↑
1	63.167	0.060	99.90	71.276	32.800	53.98	79.197	65.106	17.79	77.820	60.153	22.70
2	26.350	0.060	99.77	42.553	0.006	99.99	58.396	30.215	48.26	55.644	20.316	63.49
4	26.350	0.060	99.77	42.553	0.006	99.99	58.396	0.006	99.99	55.644	0.042	99.92
8	26.350	0.060	99.77	42.553	0.006	99.99	58.396	0.006	99.99	55.644	0.041	99.93
16	26.350	0.060	99.77	42.553	0.006	99.99	58.396	0.006	99.99	55.644	0.041	99.93
32	26.350	0.060	99.77	42.553	0.006	99.99	58.396	0.006	99.99	55.644	0.041	99.93
64	26.350	0.060	99.77	42.553	0.006	99.99	58.396	0.006	99.99	55.644	0.041	99.93
128	26.350	0.060	99.77	42.553	0.006	99.99	58.396	0.006	99.99	55.644	0.041	99.93

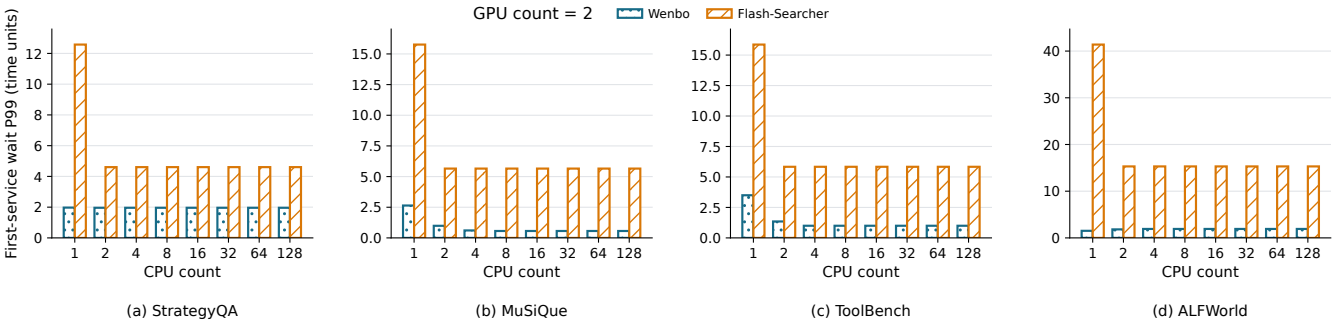


Figure 5: P99 first-service waiting time on ARGB-R as the number of CPUs varies, with the number of GPUs fixed at 2.

Table 3: P99 foreground response time (units) in the same-source mixed-workload experiment on ARGB-R, with 2 GPUs, 8 CPUs, and 16 outstanding background requests maintained throughout.

Dataset	Foreground P99 Response Time		Speedup↑
	Flash-Searcher	Wenbo	
StrategyQA	8.892	5.911	1.50×
MuSiQue	10.881	7.035	1.55×
ToolBench	11.458	7.251	1.58×
ALFWorld	29.631	18.837	1.57×

to DAG completion). We maintain 16 outstanding background requests, replenishing each upon completion. Foreground and background requests are sampled uniformly with replacement from the same workload-specific DAG set, differing only in priority. Foreground arrivals follow a Poisson process at 20% of batch-average throughput. After warming up with 1,000 foreground requests, we measure the next 10,000 and wait for all to complete.

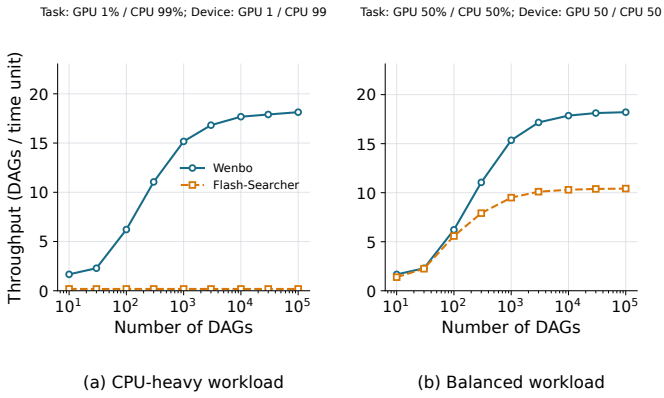
As shown in Figure 5, Wenbo substantially reduces P99 first-service waiting time across all workloads, achieving 22×–99× lower waiting times than the baseline. We also find that the magnitude of the improvement varies across datasets, reflecting differences in their task-level device-demand distributions and, consequently, in the devices required by the DAG nodes selected for first service. Table 3 further shows that Wenbo reduces P99 end-to-end response time by an average of 1.55× across all four datasets. The smaller gains suggest that subsequent execution and dependency constraints continue to contribute to tail latency. Overall, Wenbo improves both foreground responsiveness and end-to-end tail performance under mixed-priority workloads.

5.3 Evaluation on ARGB-S

We use a synthetic benchmark to isolate the effects of task count, DAG complexity, node-type distribution, and device configuration.

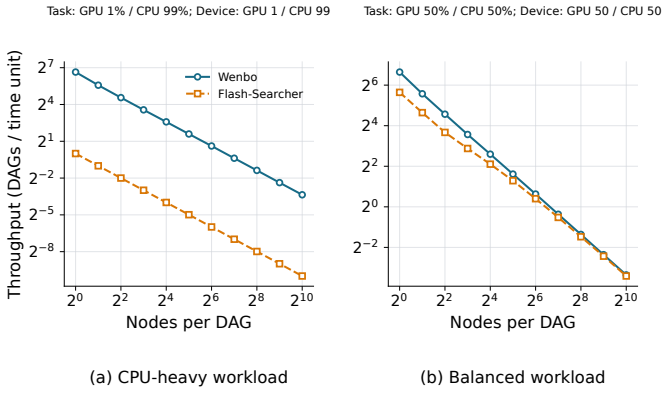
5.3.1 ***Scaling with Task Number.*** We generate synthetic DAGs that match the structural distributions of the real-world ARGB-R datasets, with 5.48 ± 2.73 nodes and a depth of 4.42 ± 2.58. We evaluate two workload configurations. The CPU-heavy workload assigns GPU and CPU node probabilities of 1% and 99%, respectively, with one GPU and 99 CPUs. The balanced workload uses equal node-type probabilities and 50 GPUs and 50 CPUs.

As shown in Figure 6(a), under the CPU-heavy workload, the throughput speedup of Wenbo over Flash-Searcher increases from 9× to 99× as the number of DAGs grows, eventually approaching the capacity gap implied by the resource ratio. On the other hand, as shown in Figure 6b), Wenbo shows a smaller but consistent throughput advantage for Wenbo under the balanced workload, with speedups increasing from 1.1× to 1.75×. The reason is that the



(a) CPU-heavy workload (b) Balanced workload

Figure 6: Throughput as the DAG number varies on ARGB-S.



(a) CPU-heavy workload (b) Balanced workload

Figure 7: Throughput as the DAG size varies on ARGB-S.

task node-type distribution better matches device availability, reducing resource waste caused by task-level resource pairing. Overall, both results align with our theoretical analysis: the maximum speedup of node-level over task-level parallelism is bounded by the smaller of the resource imbalance ratio and the workload imbalance ratio.

5.3.2 ***Impact of DAG Size.*** To evaluate the impact of task complexity, we fix the number of inference tasks at 1,000 and vary the number of nodes per DAG, while keeping the workload distributions and device configurations unchanged from the previous experiment.

As shown in Figure 7, although throughput decreases for both methods as DAG size increases, Wenbo maintains an approximately 99× throughput advantage over Flash-Searcher. This result is consistent with our theoretical analysis of the upper bound on the speedup of node-level over task-level parallelism. However, we also notice that under the balanced workload (Figure 7b)), the performance advantage of Wenbo is less pronounced and diminishes as DAGs grow. This is because balanced tasks and devices leave less room for improvement through node-level parallelism. Moreover, larger DAGs expose more executable nodes, allowing Flash-searcher to saturate the available devices and thereby reducing the marginal benefit of Wenbo's scheduling strategy.

6 Related Work

Wenbo lies at the intersection of four three lines: agent reasoning systems, DAG-based LLM reasoning, and heterogeneous resource scheduling. Existing approaches address these aspects largely in isolation, leaving open the challenge of efficiently executing large numbers of concurrent agent workflows with dynamic dependencies and heterogeneous CPU/GPU requirements.

6.1 Agent Reasoning Systems

Frameworks such as LangChain, AutoGen, and BabyAGI support agent composition, multi-agent interaction, and LLM-driven task planning [3, 8, 15]. However, these frameworks primarily focus on orchestration and functionality, rather than efficient resource utilization at scale. Distributed execution frameworks such as Ray and agent-serving systems such as AgentServe provide infrastructure for parallel task execution [7]. However, coarse-grained task-level resource allocation can cause resource fragmentation because agent workflows interleave GPU-intensive LLM inference with CPU-intensive operations such as tool execution and memory retrieval. GPUs may remain idle during CPU-bound phases, while CPU capacity is underutilized during GPU-bound phases. Techniques such as continuous batching and prefill–decode disaggregation improve LLM serving efficiency, but do not by themselves address resource fragmentation across an entire agent workflow [1, 6]. In contrast, Wenbo decomposes agent workflows into fine-grained DAG nodes and schedules ready operations across a shared CPU/GPU worker pool.

6.2 DAG-based LLM Reasoning

Tree of Thoughts explores alternative reasoning paths through tree-based search, while Graph of Thoughts generalizes reasoning to graph structures that support more flexible composition of intermediate states [2, 17]. These approaches focus on improving reasoning quality rather than scheduling execution across concurrent workflows. DAGs are also used in LLM serving and distributed model execution. Sarathi-Serve investigates scheduling strategies for LLM inference, while Alpa decomposes model computation to exploit parallelism across devices [1, 19]. Unlike these systems, which primarily target model execution or serving pipelines, agent execution DAGs combine heterogeneous operations whose dependencies may evolve at runtime. Their nodes vary in granularity and resource demand, requiring coordination across both CPU and GPU workers. Wenbo addresses these challenges by dynamically scheduling ready nodes across concurrent agent workflows.

6.3 Heterogeneous Resource Scheduling

Prior work has extensively studied resource allocation in cluster computing and deep learning. TetriSched uses mixed-integer linear programming to schedule jobs with diverse resource requirements, Gandiva improves GPU utilization through time-slicing and migration, and Optimus dynamically adjusts resource allocation

based on predicted job performance [9, 14, 16]. These systems primarily schedule jobs or training tasks, rather than fine-grained operations from dynamically evolving agent workflows. Model-parallel systems such as Alpa operate at a finer granularity by decomposing model computation into operators and mapping them across devices [19]. However, existing methods mainly focus on parallelizing model execution rather than coordinating heterogeneous operations across many independent workflows. Wenbo combines fine-grained execution with cross-workflow scheduling: a global scheduler dispatches dependency-ready nodes to appropriate CPU or GPU workers, exploiting parallelism both within and across agent trajectories.

In summary, Wenbo combines DAG-based execution with fine-grained, cross-workflow CPU/GPU scheduling for dynamically evolving agent workloads. Its focus is to improve aggregate throughput by exploiting parallelism across both operations within a workflow and independent agent trajectories.

7 Conclusion

We presented **Wenbo**, a DAG-centric execution system for scaling LLM-based agent reasoning. Wenbo decouples reasoning orchestration from resource management by incrementally materializing dynamic reasoning graphs into executable DAGs and scheduling individual computation steps across heterogeneous GPU and CPU resources. We further developed a theoretical framework to characterize the throughput benefits of DAG-level scheduling and evaluated the system on synthetic and realistic agent workloads, demonstrating significant improvements over task-level scheduling. Our work highlights DAG-centric execution as a promising approach to reducing resource fragmentation and improving the scalability of large-scale agent services.

References

[1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).

[2] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 17682–17690.

[3] Harrison Chase. 2022. LangChain. https://github.com/langchain-ai/langchain.

[4] Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics* 9 (2021), 346–361.

[5] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 443–462.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[7] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging AI applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 561–577.

[8] Yohei Nakajima. 2023. BabyAGI: An Autonomous AI Agent Framework. *Online]. Available: https://github.com/yoheinakajima/babyagi* (2023).

[9] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. 2018. Optimus: an efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*. 1–14.

[10] Tianrui Qin, Qianben Chen, Sinuo Wang, He Xing, He Zhu, Dingfeng Shi, Xinxin Liu, Ge Zhang, Jiaheng Liu, Xitong Gao, et al. 2026. Flash-searcher: Fast and effective web agents via dag-based parallel execution. In *International Conference on Learning Representations*, Vol. 2026. 45026–45066.

[11] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. *URL https://arxiv.org/abs/2307.16789* (2023).

[12] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2020. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768* (2020).

[13] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. ♫ musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.

[14] Alexey Tumanov, Timothy Zhu, Jun Woo Park, Michael A Kozuch, Mor Harchol-Balter, and Gregory R Ganger. 2016. TetriSched: global rescheduling with adaptive plan-ahead in dynamic heterogeneous clusters. In *Proceedings of the Eleventh European Conference on Computer Systems*. 1–16.

[15] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023).

[16] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. 2018. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 595–610.

[17] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.

[18] Yuanhe Zhang, Ilja Kuzborskij, Jason Lee, Chenlei Leng, and Fanghui Liu. 2026. DAG-Math: Graph-of-thought guided mathematical reasoning in LLMs. In *International Conference on Learning Representations*, Vol. 2026. 74347–74389.

[19] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating Inter-and Intra-Operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.